HCL TRAINING PROJECT REPORT

STOCKSMART

Retail Inventory Optimization System

Submitted by

Ananya Agrawal

Details	
Course	B.Tech – Computer Science and Engineering
College	ABES Engineering College, Ghaziabad
Training Organization	HCL
Training / Project	HCL Training Project
Training Duration	September 2026
Academic Year	2026-27

Declaration

I hereby declare that this report, titled “StockSmart: Retail Inventory Optimization System”, presents the work carried out by me as part of the HCL Training Project. The work described in this report was completed by me during the training programme, and the report accurately describes the functionality that has been implemented in the project.

All sources of help and guidance received during the project have been acknowledged. This report has not been submitted, in whole or in part, for any other purpose.

Field	**Details**
Name	Ananya
Registration / Roll No.	2400320100187
Course	B.Tech – Computer Science and Engineering
College	ABES Engineering College, Ghaziabad
Date	30th September, 2026
Signature	

Certified by	**Details**
Trainer / Mentor	
Signature	

Acknowledgement

I express my sincere gratitude to HCL for providing the training programme and the opportunity to work on the StockSmart project. The programme gave me practical exposure to backend development using Java, Spring Boot, Spring Data JPA, Hibernate and MySQL.

I thank my trainers and mentors, [Trainer Name, if required], for their guidance, clear explanations and timely feedback during the training. Their support helped me understand layered application design, database mapping and API development, and helped me resolve the technical issues that arose during development.

I also thank ABES Engineering College, Ghaziabad, and my faculty for their encouragement, and my peers and family for their support throughout the project.

Ananya Agrawal

Table of Contents

Declaration....1
Acknowledgement....2
Table of Contents....3
Chapter 1 – Introduction....5
1.1 Project Overview....5
1.2 Problem Statement....5
1.3 Objectives....5
1.4 Scope....6
Chapter 2 – Topics Covered....6
Chapter 3 – Project Description....7
Chapter 4 – System Requirements....7
4.1 Hardware Requirements....7
4.2 Software Requirements....8
Chapter 5 – System Design....8
5.1 System Architecture....8
5.2 Project Structure....9
5.3 Layered Architecture....9
Chapter 6 – Modules....11
6.1 Product Management....11
6.2 Supplier Management....11
6.3 Location Management....11
6.4 Inventory Management....12
Chapter 7 – Database Design....13
7.1 Database Overview....13
7.2 Products Table....13
7.3 Suppliers Table....13
7.4 Locations Table....14
7.5 Inventory Table....14
7.6 Entity Relationships....14
7.7 ER Diagram....15
Chapter 8 – Features....15
8.1 Core Features....15
8.2 Unique Features....16
Chapter 9 – API Documentation....17
9.1 Representative Examples....18
Chapter 10 – Implementation....18

10.1 Error Handling and Validation 19
Chapter 11 – Testing 20
Chapter 12 – Results 21
Chapter 13 – Challenges 22
Chapter 14 – Future Enhancements 22
Chapter 15 – Conclusion 23
References 23

(Right-click the table and choose “Update Field” in Microsoft Word to refresh page numbers.)

Chapter 1 – Introduction

1.1 Project Overview

StockSmart is a backend retail inventory management system developed using Java, Spring Boot, Spring Data JPA, Hibernate and MySQL. It provides a centralized backend for managing products, suppliers, retail and warehouse locations, and the quantity of each product held at each location. The system exposes RESTful APIs that were tested using Postman.

The application follows a layered architecture in which HTTP requests pass from the controller layer to the service layer, then to the repository layer, and finally to the MySQL database through Spring Data JPA and Hibernate.

A key design decision is that stock quantity is not stored inside the Product entity. Stock is held in a separate Inventory entity that links one Product to one Location. This allows the same product to have different quantities at different locations, as shown below.

Product	Location	Quantity
iPhone 16	Noida Central Warehouse	50 units
iPhone 16	Ghaziabad Store	12 units

Table 1.1: Example of location-wise inventory

1.2 Problem Statement

Retail businesses usually hold stock across several stores and warehouses. When product data, supplier data and stock quantities are kept in separate or informal records, it is difficult to know how much stock of a product is available at each location. A system that stores stock as a single figure per product cannot represent this.

StockSmart addresses this by providing a centralized backend that keeps product, supplier and location records and stores stock separately for each product-location combination.

1.3 Objectives

- Develop a centralized retail inventory management backend.
- Manage product information efficiently.
- Maintain supplier records.
- Manage multiple stores and warehouse locations.
- Track product quantity at individual locations.
- Provide RESTful APIs for inventory operations.
- Persist application data in a MySQL database.
- Apply layered architecture using Spring Boot.
- Demonstrate the practical use of CRUD operations, JPA, Hibernate and REST APIs.
- Create a foundation that can later be extended into an inventory optimization platform.

1.4 Scope

Current Scope (Implemented)

- Product management
- Supplier management
- Location management
- Location-specific inventory
- CRUD REST APIs
- MySQL persistence
- Backend service architecture
- API testing using Postman

Future Scope (Not Implemented)

The following items are not part of the current system and are listed only as possible future work: authentication and authorization, role-based access control, low-stock alerts, automatic reorder recommendations, stock transfer between locations, purchase order management, sales integration, inventory analytics, demand forecasting, dashboard, reporting and a notification system. See Chapter 14.

Chapter 2 – Topics Covered

The following topics were covered during the training and applied in this project.

Topic	Application in the Project
Java	Used as the programming language for all application code.
Object-Oriented Programming	Applied through entity classes, service classes, encapsulation and separation of responsibilities.
Spring Boot	Used to create and configure the application, with an embedded server and auto-configuration.
REST API Development	Controllers expose endpoints using HTTP methods (POST, GET, PUT, DELETE) for each module.
Spring Data JPA	Repository interfaces provide database access without hand-written SQL.
Hibernate	Acts as the JPA implementation that generates SQL and maps objects to tables.
ORM	Java entity classes are mapped to the products, suppliers, locations and inventory tables.
MySQL	Relational database (MySQL 8.0, database hcl) used for persistent storage.
SQL / Database Concepts	Primary keys, foreign keys, unique constraints and NOT NULL constraints were used in the schema.
Maven	Used for dependency management and project build.
CRUD Operations	Create, read, update and delete operations implemented for each module.
Layered Architecture	Code is separated into controller, service, repository and entity packages.

Topic	Application in the Project
Entity Relationships	@ManyToOne relationships link Inventory to Product and to Location.
Postman API Testing	Used to send requests to the APIs and verify responses.
Backend Development	The whole project is a server-side application with no frontend.
Database Design	Tables, keys and relationships were designed so that stock is stored per product and location.

Table 2.1: Training topics and their application

Chapter 3 – Project Description

StockSmart is a backend application with four implemented modules: Product Management, Supplier Management, Location Management and Inventory Management. Each module has an entity class, a repository interface, a service class and a REST controller.

Module	Description
Product Management	Stores product details such as name, SKU, category, price and reorder level.
Supplier Management	Stores supplier contact details such as name, email, phone and address.
Location Management	Stores stores and warehouses with a name, type and address.
Inventory Management	Stores the quantity of a product at a location by linking a Product and a Location.

Table 3.1: Implemented modules

The data flow is as follows. A client such as Postman sends an HTTP request to a controller. The controller passes the request to the service, which applies the business logic and calls the repository. The repository uses Spring Data JPA and Hibernate to read from or write to the MySQL database. The result is returned to the client as a JSON response.

The application runs on port 8081 and connects to a MySQL database named hcl. Tables are created and updated automatically by Hibernate.

Note: The current implementation is a backend only. It does not include a frontend, dashboard, authentication, notifications or any optimization algorithm. The name “Optimization System” describes the intended long-term direction of the project; the current system provides the data foundation for it.

Chapter 4 – System Requirements

4.1 Hardware Requirements

- Laptop or desktop computer
- Minimum 8 GB RAM recommended
- Sufficient free storage for the JDK, MySQL, Maven dependencies and project files
- Internet connection for development and dependency downloads

4.2 Software Requirements

Component	Details
Operating System	Windows, Linux or macOS
Java	Java JDK (Maven configured for release 17; developed and run on Java 25.0.2)
Framework	Spring Boot 4.1.1
Persistence	Spring Data JPA with Hibernate
Build Tool	Maven
Database	MySQL 8.0
IDE	Visual Studio Code
API Testing	Postman

Table 4.1: Software requirements

Chapter 5 – System Design

5.1 System Architecture

StockSmart uses a layered architecture. Each layer has a single responsibility and communicates only with the layer next to it.

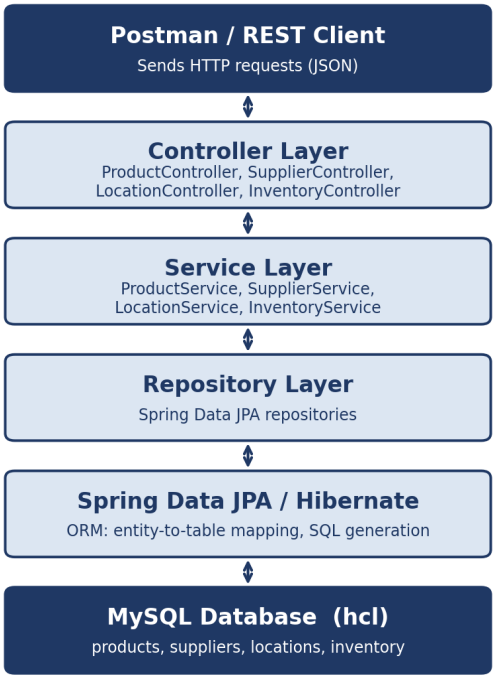


Figure 5.1: System architecture of StockSmart

5.2 Project Structure

The source code is located under src/main/java/com/hcl/StockSmart/ and is organised into four packages, in addition to the main application class.

```
StockSmart/
├── StockSmartApplication.java
├── controller/
│   ├── ProductController.java
│   ├── SupplierController.java
│   ├── LocationController.java
│   └── InventoryController.java
├── service/
│   ├── ProductService.java
│   ├── SupplierService.java
│   ├── LocationService.java
│   └── InventoryService.java
├── repository/
│   ├── ProductRepository.java
│   ├── SupplierRepository.java
│   ├── LocationRepository.java
│   └── InventoryRepository.java
└── entity/
    ├── Product.java
    ├── Supplier.java
    ├── Location.java
    └── Inventory.java
```

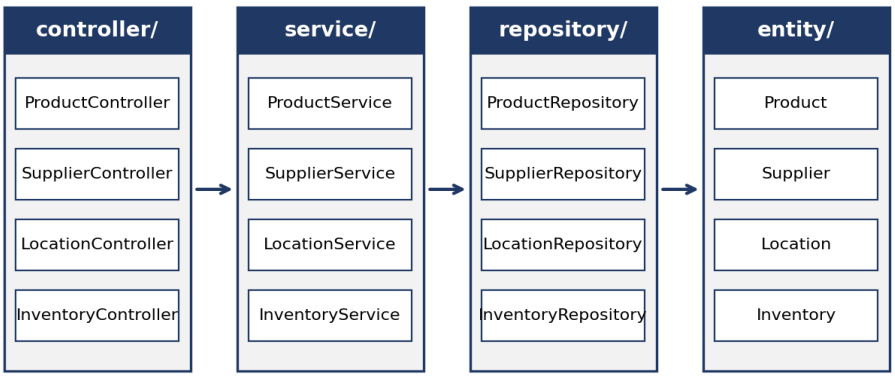


Figure 5.2: Package structure and layer dependencies

5.3 Layered Architecture

Layer	Responsibility
Client	Postman or any REST client sends HTTP requests with JSON or query parameters.
Controller	Handles HTTP requests, maps endpoints, and returns HTTP responses.
Service	Contains business logic. It checks that the required Product or Location exists and coordinates the repositories.
Repository	Provides database access through Spring Data JPA interfaces.
Entity	Java classes mapped to database tables, including the relationships between them.
Database	MySQL database hcl, which stores the persistent application data.

Table 5.1: Responsibilities of each layer

Configuration. The application is configured in application.properties. Credentials are shown as placeholders for security.

```
spring.application.name=StockSmart
spring.datasource.url=jdbc:mysql://localhost:3306/hcl
spring.datasource.username=<username>
spring.datasource.password=<password>
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
server.port=8081
```

Hibernate (through JPA) performs object-relational mapping, and ddl-auto=update creates or updates the tables from the entity classes. The show-sql setting prints the generated SQL statements to the console. Note: the project's source tree also contains an unused model folder, which is not part of the implemented application layers.

Chapter 6 – Modules

6.1 Product Management

The Product module manages the catalogue of items sold. Each product has a name, a unique SKU (stock keeping unit), a category, a price and a reorder level. The reorder level records the minimum desired stock; in the current version it is stored only, and no logic uses it yet.

Method	Endpoint	Purpose
POST	`/api/products`	Create a new product
GET	`/api/products`	Retrieve all products
GET	`/api/products/{id}`	Retrieve a product by ID
PUT	`/api/products/{id}`	Update an existing product
DELETE	`/api/products/{id}`	Delete a product

Table 6.1: Product APIs

Example request body for POST /api/products:

```
{
  "name": "iPhone 16",
  "sku": "IPH16-001",
  "category": "Electronics",
  "price": 79999,
  "reorderLevel": 10
}
```

6.2 Supplier Management

The Supplier module maintains records of suppliers, including name, email, phone and address. It provides full CRUD operations. Supplier records are currently independent: no Product-Supplier relationship exists in the code, so a product cannot yet be linked to a supplier.

Method	Endpoint	Purpose
POST	`/api/suppliers`	Create a new supplier
GET	`/api/suppliers`	Retrieve all suppliers
GET	`/api/suppliers/{id}`	Retrieve a supplier by ID
PUT	`/api/suppliers/{id}`	Update an existing supplier
DELETE	`/api/suppliers/{id}`	Delete a supplier

Table 6.2: Supplier APIs

6.3 Location Management

Locations represent the places where stock is held. In a retail business, stock is kept in several stores and warehouses, and each has its own quantity of every product. Managing locations as separate records makes it possible to track stock at each place. Each location has a name, a type (for example WAREHOUSE or STORE) and an address.

Method	Endpoint	Purpose
POST	`/api/locations`	Create a new location
GET	`/api/locations`	Retrieve all locations
GET	`/api/locations/{id}`	Retrieve a location by ID
PUT	`/api/locations/{id}`	Update an existing location
DELETE	`/api/locations/{id}`	Delete a location

Table 6.3: Location APIs

Locations used during testing:

Name	Type	Address
Noida Central Warehouse	WAREHOUSE	Noida, Uttar Pradesh
Ghaziabad Store	STORE	Ghaziabad, Uttar Pradesh

Table 6.4: Sample locations

6.4 Inventory Management

The Inventory module records how many units of a product are held at a location. Each Inventory record references one Product and one Location using @ManyToOne relationships and stores a quantity. A unique constraint on product_id and location_id is defined in the database so that the same product-location pair is not intended to have more than one record. The InventoryRepository contains the method findByProductAndLocation(Product product, Location location) to look up the record for a given pair.

Method	Endpoint	Purpose
POST	`/api/inventory?productId={productId}&locationId={locationId}&quantity={quantity}`	Create an inventory record

Method	Endpoint	Purpose
GET	/api/inventory	Retrieve all inventory records
GET	/api/inventory/{id}	Retrieve an inventory record by ID
PUT	/api/inventory/{id}?quantity={quantity}	Update the quantity
DELETE	/api/inventory/{id}	Delete an inventory record

Table 6.5: Inventory APIs

1. Receive the product ID, location ID and quantity.
2. Verify that the product exists.
3. Verify that the location exists.
4. Create the inventory record.
5. Save the record through the JPA repository.
6. Return the inventory information.

Examples of location-wise inventory creation:

```
POST http://localhost:8081/api/inventory?productId=2&locationId=1&quantity=50
POST http://localhost:8081/api/inventory?productId=2&locationId=2&quantity=12
```

The two requests store the same product (ID 2) at two different locations with different quantities.

Chapter 7 – Database Design

7.1 Database Overview

The application uses a MySQL 8.0 database named hcl with four tables: products, suppliers, locations and inventory. The tables are generated by Hibernate from the entity classes. The data types below are those derived by JPA/Hibernate for MySQL; exact lengths (for example VARCHAR(255)) follow the default Hibernate mapping.

7.2 Products Table

Column	Type	Constraint	Description
id	BIGINT	PK	Product identifier
name	VARCHAR	NOT NULL	Product name
sku	VARCHAR	UNIQUE, NOT NULL	Stock keeping unit

Column	Type	Constraint	Description
category	VARCHAR	-	Product category
price	DOUBLE	-	Product price
reorder_level	INT	-	Minimum desired stock

Table 7.1: products

7.3 Suppliers Table

Column	Type	Constraint	Description
id	BIGINT	PK	Supplier identifier
name	VARCHAR	NOT NULL	Supplier name
email	VARCHAR	-	Supplier email address
phone	VARCHAR	-	Supplier phone number
address	VARCHAR	-	Supplier address

Table 7.2: suppliers

7.4 Locations Table

Column	Type	Constraint	Description
id	BIGINT	PK	Location identifier
name	VARCHAR	NOT NULL	Location name
type	VARCHAR	NOT NULL	Location type (e.g. WAREHOUSE, STORE)
address	VARCHAR	-	Location address

Table 7.3: locations

7.5 Inventory Table

Column	Type	Constraint	Description
id	BIGINT	PK	Inventory record identifier
product_id	BIGINT	FK → products.id	Product held
location_id	BIGINT	FK → locations.id	Location where stock is held
quantity	INT	-	Units in stock

Table 7.4: inventory (unique constraint on product_id + location_id)

7.6 Entity Relationships

- One Product can have many Inventory records (one for each location where it is stocked).
- One Location can have many Inventory records (one for each product stored there).
- Each Inventory record belongs to exactly one Product and one Location.
- Supplier is an independent entity. No foreign key links it to Product or any other table in the current implementation.

7.7 ER Diagram

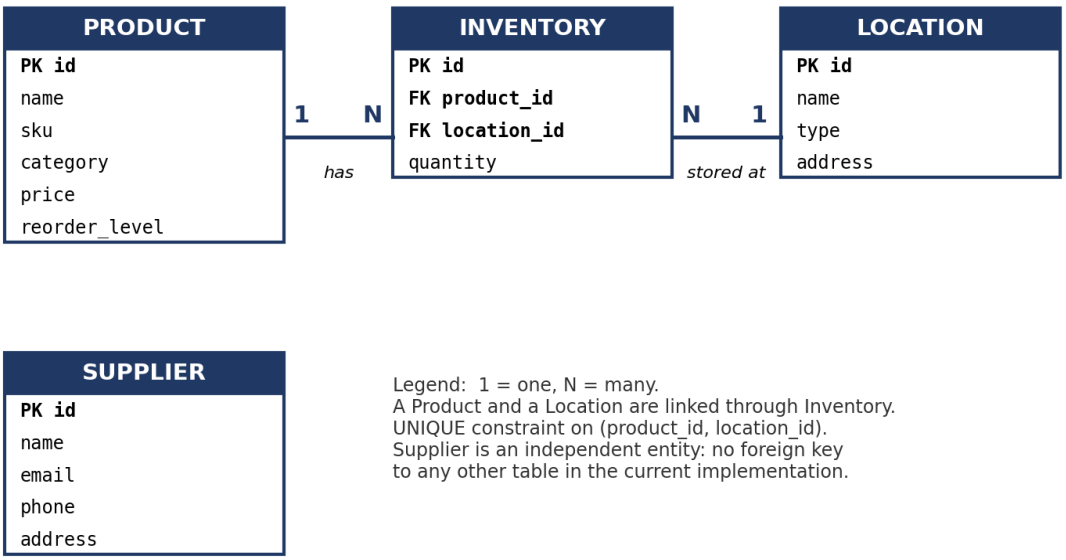


Figure 7.1: Entity-Relationship diagram of StockSmart

Chapter 8 – Features

8.1 Core Features

No.	Feature	Description
1	Product Management	Create, view, update and delete product records.
2	Supplier Management	Create, view, update and delete supplier records.
3	Location Management	Create, view, update and delete store and warehouse records.
4	Inventory Management	Create, view, update and delete inventory records.
5	CRUD Operations	Implemented for all four modules.
6	RESTful API Architecture	Resource-based endpoints under /api using standard HTTP methods.
7	MySQL Persistence	All data is stored in the MySQL database hcl.
8	Multi-Location Tracking	Stock is stored separately for each location.
9	JPA Relationships	@ManyToOne relationships from Inventory to Product and Location.
10	API Testing	APIs tested through Postman.
11	Layered Architecture	Separate controller, service, repository and entity layers.

Table 8.1: Implemented features

8.2 Unique Features

- **Multi-location inventory tracking:** the same product can have different quantities at different locations.
- **Product-location based representation:** inventory is modelled as Product + Location → Inventory instead of a single quantity on the product.
- **Centralized inventory management:** products, suppliers, locations and stock are managed by one backend.
- **Layer separation:** controller, service, repository and entity responsibilities are kept separate, which makes the code easier to maintain and extend.
- **Foundation for optimization:** the location-wise data model can support future inventory optimization features.

Chapter 9 – API Documentation

All endpoints are served from http://localhost:8081. The table below lists every implemented endpoint.

Module	Method	Endpoint	Purpose
Product	POST	`/api/products`	Create a product
Product	GET	`/api/products`	List all products
Product	GET	`/api/products/{id}`	Get a product by ID
Product	PUT	`/api/products/{id}`	Update a product
Product	DELETE	`/api/products/{id}`	Delete a product
Supplier	POST	`/api/suppliers`	Create a supplier
Supplier	GET	`/api/suppliers`	List all suppliers
Supplier	GET	`/api/suppliers/{id}`	Get a supplier by ID
Supplier	PUT	`/api/suppliers/{id}`	Update a supplier
Supplier	DELETE	`/api/suppliers/{id}`	Delete a supplier
Location	POST	`/api/locations`	Create a location
Location	GET	`/api/locations`	List all locations
Location	GET	`/api/locations/{id}`	Get a location by ID
Location	PUT	`/api/locations/{id}`	Update a location
Location	DELETE	`/api/locations/{id}`	Delete a location
Inventory	POST	`/api/inventory?productId={productId}&locationId={locationId}&quantity={quantity}`	Create an inventory record

Module	Method	Endpoint	Purpose
Inventory	GET	/api/inventory	List all inventory records
Inventory	GET	/api/inventory/{id}	Get an inventory record
Inventory	PUT	/api/inventory/{id}?quantity={quantity}	Update the quantity
Inventory	DELETE	/api/inventory/{id}	Delete an inventory record

Table 9.1: Complete API list

9.1 Representative Examples

Example 1: Create a location

POST /api/locations

```
{
  "name": "Noida Central Warehouse",
  "type": "WAREHOUSE",
  "address": "Noida, Uttar Pradesh"
}
```

Example 2: Create an inventory record

POST /api/inventory?productId=2&locationId=1&quantity=50

The response contains the saved inventory record, with its generated ID, the linked Product and Location, and the quantity. The exact response body depends on the entity serialization and is not reproduced here.

Example 3: Update inventory quantity

PUT /api/inventory/{id}?quantity=40 updates the quantity of the specified inventory record.

Chapter 10 – Implementation

1. Created a Spring Boot project using Maven.
2. Configured the MySQL database connection in application.properties.
3. Added the Spring Web dependency.
4. Configured Spring Data JPA and Hibernate.
5. Created the entity classes: Product, Supplier, Location and Inventory.
6. Created the repository interfaces for each entity.
7. Created the service classes containing the business logic.
8. Created the REST controllers to expose the endpoints.
9. Implemented CRUD operations for each module.

10. Created the entity relationships for Inventory (Product and Location).
11. Started the application on port 8081.
12. Tested the APIs using Postman.
13. Verified persistence of data in MySQL.

10.1 Error Handling and Validation

Error handling in the current version is basic and is implemented in the service layer.

- Before an inventory record is created, the service checks that the requested Product and Location exist. If not, a runtime exception is thrown with the message “Product not found” or “Location not found”.
- When a record is requested by ID and does not exist, the service throws a runtime exception with a message such as “Product not found”, “Supplier not found”, “Location not found” or “Inventory not found”.

The current version does not include a global exception handler, custom HTTP error responses, bean validation annotations or authentication. These are listed under Future Enhancements.

Screenshots of the running application and the database:

Chapter 11 – Testing

Testing tool: Postman. **Approach:** functional API testing. Each API was called with sample data and the response and database content were checked. The Product, Supplier and Location CRUD APIs were tested successfully. The Inventory test cases are marked “To be tested” because their execution has not been confirmed at the time of writing; update this column after testing.

No.	Test Case	Module	Method	Expected Result	Status
1	Create product	Product	POST	Product saved and returned	Passed
2	Get all products	Product	GET	List of products returned	Passed
3	Get product by ID	Product	GET	Matching product returned	Passed
4	Update product	Product	PUT	Product updated	Passed
5	Delete product	Product	DELETE	Product removed	Passed

No.	Test Case	Module	Method	Expected Result	Status
6	Create supplier	Supplier	POST	Supplier saved and returned	Passed
7	Get suppliers	Supplier	GET	List of suppliers returned	Passed
8	Update supplier	Supplier	PUT	Supplier updated	Passed
9	Delete supplier	Supplier	DELETE	Supplier removed	Passed
10	Create location	Location	POST	Location saved and returned	Passed
11	Get locations	Location	GET	List of locations returned	Passed
12	Update location	Location	PUT	Location updated	Passed
13	Delete location	Location	DELETE	Location removed	Passed
14	Create inventory	Inventory	POST	Inventory record saved	To be tested
15	Get inventory	Inventory	GET	Inventory records returned	To be tested
16	Update quantity	Inventory	PUT	Quantity updated	To be tested
17	Delete inventory	Inventory	DELETE	Inventory record removed	To be tested

Table 11.1: Functional test cases

Chapter 12 – Results

The StockSmart backend was implemented and runs on port 8081 with a MySQL database. The following results were achieved:

- Centralized management of products, suppliers and locations through REST APIs.
- An inventory module that stores stock for each product-location pair.
- Persistence of all data in MySQL through Spring Data JPA and Hibernate.
- A multi-location stock model that can represent different quantities of the same product at different locations.
- A layered architecture with separate controller, service, repository and entity layers.

Verified testing results: the CRUD APIs for Product, Supplier and Location were tested through Postman and worked as expected. The Inventory APIs are implemented; their test status is given in Table 11.1.

Chapter 13 – Challenges

- **Configuring Spring Boot with MySQL:** setting the datasource URL, driver and JPA properties correctly so that the application could connect to the database.
- **Understanding JPA entity mapping:** learning how annotations map Java classes and fields to tables and columns.

- **Creating relationships:** modelling Inventory as a link between Product and Location using @ManyToOne and a unique constraint.
- **Managing CRUD across layers:** keeping controller, service and repository responsibilities consistent for four modules.
- **Testing APIs with Postman:** preparing request bodies and query parameters and checking the results in the database.
- **Application configuration:** handling the server port setting and port conflicts (the application runs on port 8081).

Chapter 14 – Future Enhancements

The following are proposed FUTURE ENHANCEMENTS. None of them are implemented in the current version of StockSmart.

No.	Enhancement	Description
1	Low-stock detection	Compare inventory quantity with the product's reorderLevel.
2	Automatic reorder suggestions	Recommend quantities to reorder based on stock levels.
3	Stock transfer	Move stock between stores and warehouses.
4	Purchase order management	Create and track orders to suppliers.
5	Sales and billing integration	Reduce stock automatically when items are sold.
6	Demand forecasting	Use historical sales data to predict demand.
7	Analytics and reporting	Inventory reports and insights.
8	Admin dashboard	A user interface for managing and viewing inventory.
9	Authentication and role-based authorization	Secure the APIs and restrict access by role.
10	Notifications	Alerts for low-stock conditions.
11	Audit logs	Record who changed stock and when.
12	Cloud deployment	Deploy the application and database on a cloud platform.

Table 14.1: Proposed future enhancements

Other improvements include a Product-Supplier relationship, global exception handling and input validation.

Chapter 15 – Conclusion

StockSmart demonstrates a practical backend implementation of a retail inventory management system using Java, Spring Boot, REST APIs, Spring Data JPA, Hibernate and MySQL. The project implements product, supplier, location and inventory management with CRUD REST APIs and persistent storage.

Its main strength is the multi-location inventory model. By representing stock as a relationship between a product and a location, the system can store different quantities of the same product at each store or warehouse. The layered architecture keeps the code organised and makes it easier to extend.

The project gave practical experience in backend development, database design, ORM and API testing. The current version does not include advanced optimization algorithms or artificial intelligence; it provides a solid foundation on which the enhancements listed in Chapter 14 can be built.

References

- Spring Boot Documentation – https://spring.io/projects/spring-boot
- Spring Data JPA Documentation – https://spring.io/projects/spring-data-jpa
- Hibernate ORM Documentation – https://hibernate.org/orm/
- MySQL 8.0 Reference Manual – https://dev.mysql.com/doc/
- Apache Maven Documentation – https://maven.apache.org/
- Postman Documentation – https://learning.postman.com/